

保護多代理系統

來源：<https://codelabs.developers.google.com/codelabs/production-ready-ai-roadshow/3-securing-a-multi-agent-system/securing-a-multi-agent-system?hl=zh-tw>

步驟數：9

「使用 Google Cloud 安全性功能保護分散式多代理系統。」

目錄

1. [1. 簡介](#)
2. [2. 設定](#)
3. [3. 建立 Sensitive Data Protection 範本](#)
4. [4. 建立 Model Armor 範本](#)
5. [5. 在「檢查使用者提示」中新增 Model Armor](#)
6. [6. 在本機部署](#)
7. [7. 測試](#)
8. [8. 附錄](#)
9. [9. 結論](#)

1. 簡介

總覽

在「[建構多代理系統](#)」中，您建構了分散式課程建立者系統，而在「[從『氛圍檢查』到以資料為準的代理評估](#)」中，您瞭解如何評估系統效能。

本實驗室的重點是解決這些安全漏洞，進而強化系統。公開代理程式端點會導致這些端點成為提示詞注入、阻斷服務和其他漏洞攻擊的目標。與使用者互動的代理程式可能會處理敏感 PII，而檢索網路的代理程式則可能擷取有害內容，或成為間接提示詞注入的受害者。為防範這些威脅，您將使用 Google Cloud 安全性工具 (包括 Model Armor 和 Sensitive Data Protection) 實作縱深防禦策略，並套用安全性最佳做法，例如最小權限 IAM 和經過驗證的網路通訊。

學習內容

- **定義安全政策**：建立 Sensitive Data Protection (SDP) 範本，偵測及遮蓋個人識別資訊 (PII)。
- **整合應用程式安全防護**：修改後端，在使用者提示詞傳送至代理程式前，先使用 Model Armor 攔截並清理提示詞。
- **驗證保護措施**：部署安全應用程式並執行紅隊情境，驗證系統是否能阻擋提示詞注入和機密資料外洩。
- **實作「政策即程式碼」** (選用)：使用 Terraform 管理 Model Armor 和 SDP 範本，確保各環境的安全篩選機制/防護措施一致。

課程內容

- 如何設定 Google Cloud Sensitive Data Protection (SDP)，以識別及遮蓋私密/機密資料。
 - 如何使用 Terraform 建立及部署 Model Armor 範本。
 - 「縱深防禦」模式，可在應用程式層保護生成式 AI 代理。
 - 如何使用紅隊演練技術稽核及驗證安全控制措施。
-

2. 設定

設定

1. 請確認你已登入帳戶。執行下列指令，取得目前的 gcloud 帳戶：

```
gcloud config get-value account
```

如果尚未登入，請執行下列指令：

```
gcloud auth login --update-adc
```

2. 為 gcloud CLI 設定有效專案。執行下列指令，取得目前的 gcloud 專案：

```
gcloud config get-value project
```

```
gcloud config get-value project
```

如果未設定，請執行下列指令：

```
gcloud config set project YOUR_PROJECT_ID
```

```
gcloud config set project YOUR_PROJECT_ID
```

將 `YOUR_PROJECT_ID` 替換為專案 ID。

3. 啟用 Cloud Run、Model Armor、資料遺失防護、Artifact Registry、Cloud Build 和 IAM 憑證的 API。

```
gcloud services enable --project $(gcloud config get-value project) \
```

```
    aiplatform.googleapis.com \
    modelarmor.googleapis.com \
    dlp.googleapis.com \
    run.googleapis.com \
    artifactregistry.googleapis.com \
    cloudbuild.googleapis.com \
    iamcredentials.googleapis.com
```

4. 設定要部署 Cloud Run 服務的預設區域。

```
gcloud config set run/region us-central1
```

請務必使用 `us-central1` 存取 Model Armor，並取得一致的範例。如要查看 Model Armor 適用地區，請按[這裡](#)。

程式碼和依附元件

1. 複製範例程式碼，然後切換到專案根目錄。

```
git clone https://github.com/h3xar0n/prai-roadshow-lab-3-starter
cd prai-roadshow-lab-3-starter
```

如要啟動 Cloud Shell 工作區，請執行下列指令：

```
cloudshell workspace .
```

依序選取「Terminal」(終端機) > 「New Terminal」(新增終端機)，開啟新的終端機。

2. 在終端機中輸入下列指令，建立 `.env` 檔案：

```
echo "GOOGLE_GENAI_USE_VERTEXAI=true" > .env
echo "GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project -q)" >> .env
echo "GOOGLE_CLOUD_REGION=$(gcloud config get-value run/region -q)" >> .env
echo "GOOGLE_CLOUD_LOCATION=global" >> .env
```

在 Cloud Shell 編輯器中，依序選取「View」 > 「Toggle Hidden Files」，即可查看隱藏檔案，例如 `.env`。

3. 在終端機中輸入下列指令，安裝依附元件：

```
uv sync
```

3. 建立 Sensitive Data Protection 範本

Model Armor 的「進階」 Sensitive Data Protection 功能會整合 Cloud DLP (Sensitive Data Protection)，檢查內容並去識別化。如要使用這項功能進行遮蓋，請先建立檢查和去識別化範本，指定要轉換的*哪些*類型的私密/機密資料，以及*如何*轉換。

ID	Job Title	Phone	Comments
359740	Senior Engineer	307-964-0673	Please email them at jane@imadethisup.com
981587	VP, Engineer	713-910-6787	none
394091	Lawyer	692-398-4146	Updated phone to: 692-398-4146
986941	Senior Ops Manager	294-967-5508	none
490456	Junior Ops Manager	791-954-3281	Tried to verify account with their SSN 222-44-5555

建立檢查範本

Sensitive Data Protection 會使用 infoType 偵測工具尋找不同類型的機密資料。系統內建 [150 多種偵測工具](#)，可使用各種偵測方法，包括模式比對 (規則運算式)、字典和以內容為準的信號。對於信用卡號碼或政府核發身分證件等特定類型，系統會驗證檢查碼，以減少誤判情形，而不只是簡單的模式比對。這些偵測器涵蓋個人識別資訊 (PII)，例如姓名和地址，以及 API 金鑰或驗證權杖等憑證，有助於防止與程式碼互動或讀取程式碼的代理程式洩漏資訊。

1. 在 Google Cloud 控制台中，依序前往「安全性」 > 「Sensitive Data Protection」。
2. 在導覽選單中，依序選取「設定」 > 「範本」。
3. 按一下「建立範本」。
4. 設定範本：
 - 「範本類型」： `Inspect`
 - 範本 ID： `sensitive-data-inspector`
 - 位置類型： `Region`
 - 「Region」 (地區)： `us-central1` (這是使用 Model Armor 的必要條件)。
5. 按一下「繼續」。
6. 在「設定偵測作業」部分，點選「管理 infoType」。
7. 使用篩選器搜尋下列 **infoTypes**，並勾選每個項目旁的核取方塊：

- CREDIT_CARD_NUMBER
- GOVERNMENT_ID
- PERSON_NAME
- EMAIL_ADDRESS
- STREET_ADDRESS
- SECURITY_DATA

8. 選取其他感興趣的項目，然後按一下「完成」。
9. 右側會顯示您選取的不同類型私密資訊，以及輸入和輸出內容。

Test your template

Input sample

This demo will detect and redact sensitive information as you type it.

Send a note to test@example.com.
 AWSAccessKeyId=AKIAJ3M6GUIM2YA37ZZA
 AWSSecretKey=A/Vd+qGojOSri8kut28nAeB0m/aLjp3loH3BesU4
 Spain Social Security Number 281234567840
 A White house is at 1600 Pennsylvania Avenue NW in Washington, D.C.
 John Smith wore pink shoes to prom.
 Mastercard Expiration Date: 01/2025

Test with sample data

Transformed sample

Send a note to [EMAIL_ADDRESS].
 [SECURITY_DATA]
 [GEOGRAPHIC_DATA] Social Security Number [GOVERNMENT_ID]
 A [GEOGRAPHIC_DATA] is at [GEOGRAPHIC_DATA]
 John Smith wore pink shoes to prom.
 Mastercard Expiration Date: [CREDIT_CARD_DATA]

10. 檢查產生的資料表，確認已加入所有這些 infoType，然後按一下「建立」。

建立去識別化範本

現在要建立去識別化範本，指定如何轉換機密資料發現項目。

Sensitive Data Protection 支援多種轉換方法，您可能想完全遮蓋街道地址等 PII，並以 [REDACTED] 等預留位置取代，但對於信用卡號碼或社會安全號碼，您可能偏好以 # 等字元遮蓋，同時保留末 4 碼以供識別。如需可兼顧安全性和可用性的轉換方法完整清單，請參閱「[去識別化技術](#)」。

1. 在 Google Cloud 控制台中，依序前往「安全性」 > 「Sensitive Data Protection」。
2. 在導覽選單中，依序選取「Configuration」 > 「範本」 > 「去識別化」。
3. 按一下「建立範本」。
4. 設定範本：
 - 「範本類型」：De-identify
 - 資料轉換類型：InfoType
 - 範本 ID：sensitive-data-redactor
 - 位置類型：Region
 - 「Region」(地區)：us-central1 (這是使用 Model Armor 的必要條件)。
5. 按一下「繼續」。
6. 在「設定去識別化」部分，您將定義多項規則。特定 infoType 的規則會覆寫預設規則。
7. 設定第一條轉換規則：
 - 轉換：Mask with character
 - 遮蓋字元：#
 - 「要忽略的字元」 > 「指定要忽略的字元」：US Punctuation...
 - 要遮蓋的字元數：12
 - 要轉換的 infoType：Specific infoTypes
 - 按一下「管理 infoType」
 - 搜尋並勾選「CREDIT_CARD_NUMBER」旁的方塊
 - 按一下 [完成]。
 - 查看輸入範例和轉換後的範例，確認只有末四碼未遮蓋，因為您選取忽略 -，並專注於 16 位數卡號的前 12 個字元。
8. 按一下「+ 新增轉換規則」，然後設定：
 - 轉換：Replace
 - 「Replace Type」(更換類型)：String
 - 字串值：[redacted] (或您想使用的任何其他字串)
 - 要轉換的 infoType：Any detected infoTypes...
9. 按一下「建立」即可儲存去識別化範本。
10. 按一下「測試」，然後選取先前建立的檢查範本，結尾為 /sensitive-data-inspector。這項測試會將檢查範本中的 infoType 與去識別化範本中的轉換作業合併。

← De-identify template details

EditCopyDelete

Configuration

Test

Inspection template (optional)

If you want to test this de-identify template with your own custom inspection template, enter your inspection template's full resource name. If you don't provide an inspection template, this de-identify template is tested with a default set of infoType detectors. [Learn more](#)

Template name

projects/aron-l200-primus/locations/us-central1/inspectTeiView details

Please provide the fully qualified resourced ID. It is formatted as follows:
projects/projectName/(locations/locationId)/inspectTemplates/templateName

Data Type

☒ Free-form☐ Structured

Input sample

This demo will detect and redact sensitive information as you type it.

He paid with 4012-8888-8888-1881. His name is Lord Bojangles.

Transformed sample

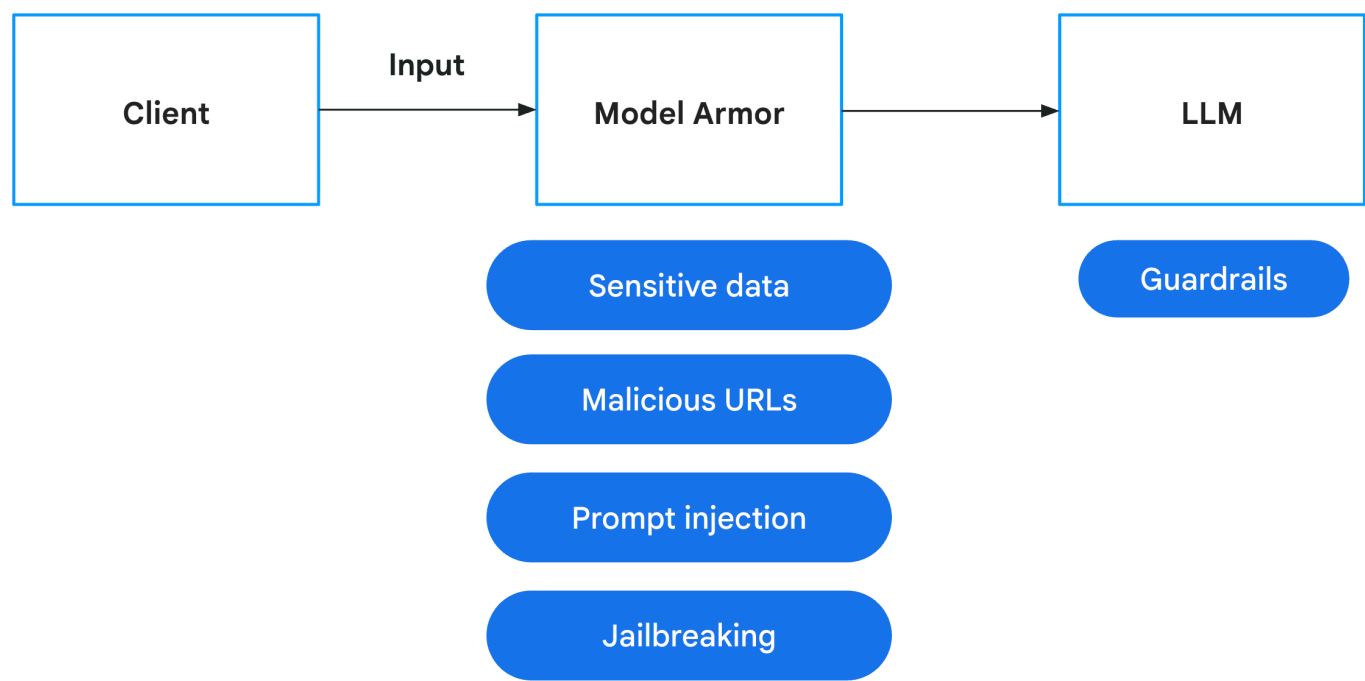
He paid with #####-#####-#####-1881. His name is [above your clearance level 😊].

這些範本現在可由 Model Armor 叫用。如要深入瞭解如何使用 Sensitive Data Protection 執行各種作業 (從每週儲存空間掃描到 BigQuery 稽核)，以及如何測試圖片和 CSV 等不同檔案類型，請參閱「[Securing Data Used for AI Applications](#)」實驗室。

如要使用 Terraform 建立這些 SDP 範本，請參閱本實驗室的附錄部分。

4. 建立 Model Armor 範本

現在要建立 Model Armor 範本，使用您剛建立的 SDP 範本處理機密資料。



Model Armor 是一項全方位的安全防護服務，可保護 Google Cloud 上的 AI 應用程式和模型。Model Armor 不會讓模型暴露於惡意輸入內容，而是充當智慧型防火牆，即時分析提示和回覆，偵測並封鎖威脅，避免造成損害。以下是 Model Armor 有助於降低的主要風險：

風險	緩解措施
提示詞注入和越獄 ：惡意使用者會設計提示詞來規避安全防護措施，試圖生成有害或非預期的內容。	建立並套用 Model Armor 安全性政策，自動偵測並阻擋提示詞注入和越獄活動。
惡意網址 ：使用者在提示中嵌入惡意連結，藉此執行有害動作或竊取資料。	設定安全政策，偵測並封鎖使用者提示中的惡意網址。
機密資料外洩 ：模型在回覆中揭露個人識別資訊 (PII)，造成隱私權侵害。	導入資料遺失防護政策，檢查提示和回覆內容，在機密資訊傳送給使用者前偵測並封鎖。

- 在 Google Cloud 控制台，使用頂端的搜尋列搜尋並前往 **Model Armor**。
- 按一下「建立範本」，然後設定下列選項：
 - 範本 ID： `course-creator-security-policy`
 - 位置類型： `Region`
 - Region (區域)： `us-central1`

- 在「偵測」下方：
 - 檢查「惡意網址偵測」
 - 勾選「提示詞注入和越獄偵測」，並將「信心水準」設為「低與以上」。
 - 勾選「Sensitive Data Protection」。
 - 將「偵測類型」設為「進階」。
 - 在「檢查範本名稱」欄位中，輸入檢查範本的完整資源名稱 (將 [YOUR_PROJECT_ID] 替換為專案 ID)： `projects/[YOUR_PROJECT_ID]/locations/us-central1/inspectTemplates/sensitive-data-inspector`
 - 在「去識別化範本名稱」欄位中，輸入去識別化範本的完整資源名稱 (將 [YOUR_PROJECT_ID] 替換為您的專案 ID)： `projects/[YOUR_PROJECT_ID]/locations/us-central1/deidentifyTemplates/sensitive-data-redactor`
- 在「負責任的 AI 技術」下方，設定：
- 仇恨言論：中等以上
- 騷擾：低於和高於
- 你選擇的所有其他項目
- 在「設定記錄功能」下方，勾選 `Prompts and responses`

3. 點選「建立」。

在環境檔案中新增範本名稱

請務必在建立時使用 `course-creator-security-policy` 範本 ID，指令碼才能正常運作。在控制台建立範本後，您需要在 `.env` 檔案中加入範本的完整資源名稱，以便載入環境以進行部署步驟。

在終端機中輸入下列指令：

```
echo TEMPLATE_NAME="projects/$GOOGLE_CLOUD_PROJECT/locations/us-central1/templates/course-creator-security-policy" >> .env
```

如要使用 Terraform 建立這個 Model Armor 範本，請參閱本實驗室的附錄部分。

步驟 5 · 約 0 分鐘

5. 在「檢查使用者提示」中新增 Model Armor

建立 Model Armor 範本後，下一步是在應用程式中強制執行這項政策。我們會修改後端，攔截使用者輸入內容，並根據安全篩選器驗證內容。確保在代理程式處理惡意提示或私密資料之前，就能在「前門」攔截。

如果您不想手動套用這些變更，而是想直接取得已完成、經過測試且穩定的程式碼，請參閱本實驗室的附錄部分。

新增依附元件

首先，我們需要在後端應用程式中新增 `google-cloud-modelarmor` 程式庫。

檔案：`app/pyproject.toml`

將 `google-cloud-modelarmor` 新增至 `dependencies` 清單：

```
[project]
# ... (existing config)
dependencies = [
    "uvicorn==0.40.0",
    "fastapi==0.123.*",
    "httpx==0.28.*",
    "httpx_sse==0.4.*",
    "google-genai==1.57.*",
    "google-cloud-logging==3.13.0",
    "opentelemetry-exporter-gcp-trace==1.11.0",
    "google-cloud-modelarmor==0.4.0", # <--- NEW DEPENDENCY
]
```

...

建立安全公用程式

在工作 1 中，請前往 `app/safety_util.py`，我們將處理及剖析 Model Armor 回覆。這樣可確保主要應用程式邏輯保持乾淨。

檔案：`app/safety_util.py`

```

# Copyright 2025 Google LLC
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

"""Utility functions for Model Armor."""

import logging
from typing import Any

from google.cloud.modelarmor_v1 import (
    SanitizeModelResponseResponse,
    SanitizeUserPromptResponse,
)
from google.cloud.modelarmor_v1.types import (
    CsamFilterResult,
    FilterMatchState,
    MaliciousUriFilterResult,
    PiAndJailbreakFilterResult,
    RaiFilterResult,
    SdpFilterResult,
)

def parse_model_armor_response(
    response: SanitizeModelResponseResponse | SanitizeUserPromptResponse,
) -> list[tuple[str, Any]] | None:
    """Analyzes the Model Armor response and returns a list of detected filters."""
    sanitization_result = response.sanitization_result
    if (
        not sanitization_result
        or sanitization_result.filter_match_state
        == FilterMatchState.NO_MATCH_FOUND
    ):
        return None

    detected_filters = []
    filter_matches = sanitization_result.filter_results

    # Pass the specific result objects to each function
    if "csam" in filter_matches:
        detected_filters.extend(
            parse_csam_filter(filter_matches["csam"].csam_filter_filter_result)
        )

```

```

if "malicious_uris" in filter_matches:
    detected_filters.extend(
        parse_malicious_uris_filter(
            filter_matches["malicious_uris"].malicious_uri_filter_result
        )
    )
if "rai" in filter_matches:
    detected_filters.extend(
        parse_rai_filter(filter_matches["rai"].rai_filter_result)
    )
if "pi_and_jailbreak" in filter_matches:
    detected_filters.extend(
        parse_pi_and_jailbreak_filter(
            filter_matches[
                "pi_and_jailbreak"
            ].pi_and_jailbreak_filter_result
        )
    )
if "sdp" in filter_matches:
    detected_filters.extend(
        parse_sdp_filter(filter_matches["sdp"].sdp_filter_result)
    )
logging.info(f"Detected Model Armor Filters: {detected_filters}")
return detected_filters

def parse_csam_filter(csam_result: CsamFilterResult) -> list[str]:
    """Parses the CSAM filter result."""
    if csam_result.match_state == FilterMatchState.MATCH_FOUND:
        return ["CSAM"]
    return []

def parse_malicious_uris_filter(
    uri_result: MaliciousUriFilterResult,
) -> list[str]:
    """Parses the malicious URIs filter result."""
    if uri_result.match_state == FilterMatchState.MATCH_FOUND:
        return ["Malicious URIs"]
    return []

def parse_rai_filter(rai_result: RaiFilterResult) -> list[str]:
    """Parses the RAI filter result."""
    if rai_result.match_state == FilterMatchState.MATCH_FOUND:
        return [
            filter_name
            for filter_name, matched in rai_result.rai_filter_type_results.items()
            if matched.match_state == FilterMatchState.MATCH_FOUND
        ]
    return []

```

```

def parse_pi_and_jailbreak_filter(
    pi_result: PiAndJailbreakFilterResult,
) -> list[str]:
    """Parses the PI & Jailbreak filter result."""
    if pi_result.match_state == FilterMatchState.MATCH_FOUND:
        return ["Prompt Injection and Jailbreaking"]
    return []

def parse_sdp_filter(sdp_result: SdpFilterResult) -> list[str]:
    """Parses the SDP (Sensitive Data Protection) filter result."""
    detected_filters = []

    inspect_result = sdp_result.inspect_result
    if (
        inspect_result
        and inspect_result.match_state == FilterMatchState.MATCH_FOUND
    ):
        for finding in inspect_result.findings:
            info_type = finding.info_type.replace("_", " ").capitalize()
            detected_filters.append(info_type)

    deidentify_result = sdp_result.deidentify_result
    if (
        deidentify_result
        and deidentify_result.match_state == FilterMatchState.MATCH_FOUND
    ):
        for info_type in deidentify_result.info_types:
            formatted_info_type = info_type.replace("_", " ").capitalize()
            detected_filters.append(formatted_info_type)

    return detected_filters

```

在後端整合 Model Armor

修改主要應用程式邏輯，初始化 Model Armor 用戶端，並在將提示傳送至自動化調度管理工具 (以及任何代理程式) 之前，先清理提示。

檔案： `app/main.py`

首先，請匯入 Model Armor 和您在 `Task 1` 中建立的新 `safety_util`。`Task 2`

```

# Task 2: import Model Armor and the new safety_util
from google.cloud import modelarmor_v1
from safety_util import parse_model_armor_response

```

在 `Task 3` 中，於 `lifespan` 或全域範圍內 (`project_id` 擷取後)，初始化用戶端：

```
# Task 3: Model Armor configuration
MODEL_ARMOR_TEMPLATE = os.getenv("TEMPLATE_NAME")
model_armor_client = modelarmor_v1.ModelArmorClient(
    client_options={"api_endpoint": "modelarmor.us-central1.rep.googleapis.com"}
)
```

對於 Task 4，我們會更新 `chat_stream` 函式：

請先新增清除邏輯，再呼叫協調器或生成內容。請務必檢查縮排，並視需要參閱[完整範例](#)。

```
# Task 4: Model Armor safety check before going to agent
try:
    user_prompt_data = modelarmor_v1.DataItem(text=request.message)
    ma_request = modelarmor_v1.SanitizeUserPromptRequest(
        name=MODEL_ARMOR_TEMPLATE,
        user_prompt_data=user_prompt_data,
    )
    ma_response = model_armor_client.sanitize_user_prompt(request=ma_request)

    # Parse response using our utility
    detected_filters = parse_model_armor_response(ma_response)

    if detected_filters:
        logger.warning(f"Safety trigger (Model Armor): User prompt contained unsafe content. Risk: {detected_filters}")
        from fastapi import HTTPException
        raise HTTPException(status_code=400, detail=f"Safety error: Prompt contains forbidden content: {detected_filters}")

except Exception as e:
    # If it is the HTTP exception we just raised, re-raise it
    if "Safety error" in str(e):
        raise e

    # Otherwise log error but fail open (or closed depending on policy - here failing open for demo simplicity unless it's a critical error)
    logger.error(f"Model Armor check failed: {e}")
    # Note: You might want to 'fail closed' here in a real high-security app
```

前端錯誤處理

更新前端，妥善處理安全錯誤 (400 Bad Request)，並向使用者顯示錯誤訊息。我們日後可能會變更這項行為，改為顯示一般錯誤訊息，但目前顯示封鎖提示的原因，有助於您開始使用這項功能。

檔案：`app/frontend/app.js`

針對 Task 5，請修改 `createForm` 事件監聽器 (或對等的提交處理常式)，剖析 JSON 錯誤回應並向使用者顯示。

```
// Task 5: display error to user
if (!response.ok) {
  let errorMessage = `HTTP error! status: ${response.status}`;
  try {
    const errorData = await response.json();
    if (errorData.detail) {
      errorMessage = errorData.detail;
    }
  } catch (e) {
    console.error("Could not parse error response JSON", e);
  }
  throw new Error(errorMessage);
}
```

步驟 6 · 約 0 分鐘

6. 在本機部署

現在要在本機部署，先測試 Model Armor 是否能如預期在代理程式前運作。

請確認 `.env` 檔案包含正確的 `TEMPLATE_NAME` 變數，指向先前建立的 Model Armor 範本。

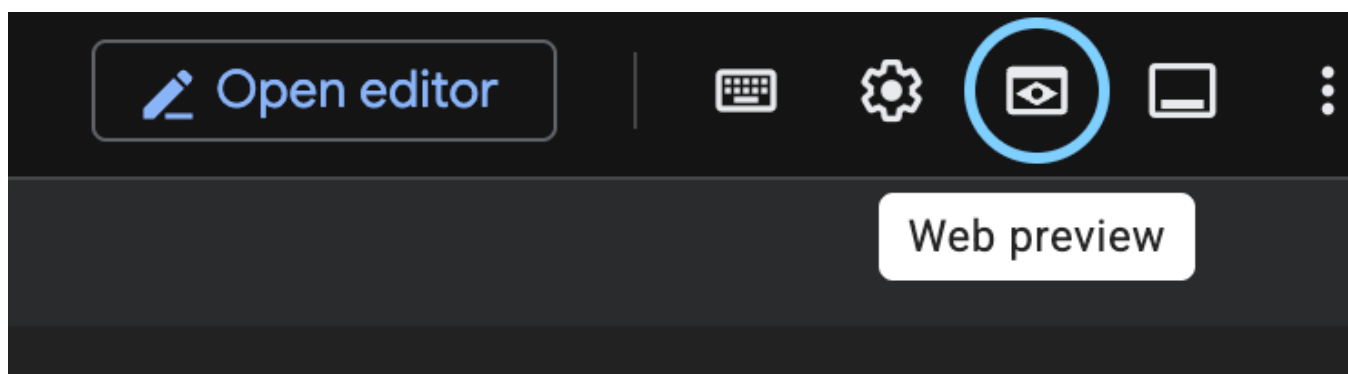
```
# Example .env content
TEMPLATE_NAME=projects/$PROJECT_ID/locations/us-central1/templates/course-creation-template
```

如要在本機部署，請使用 `run_local.sh` 指令碼：

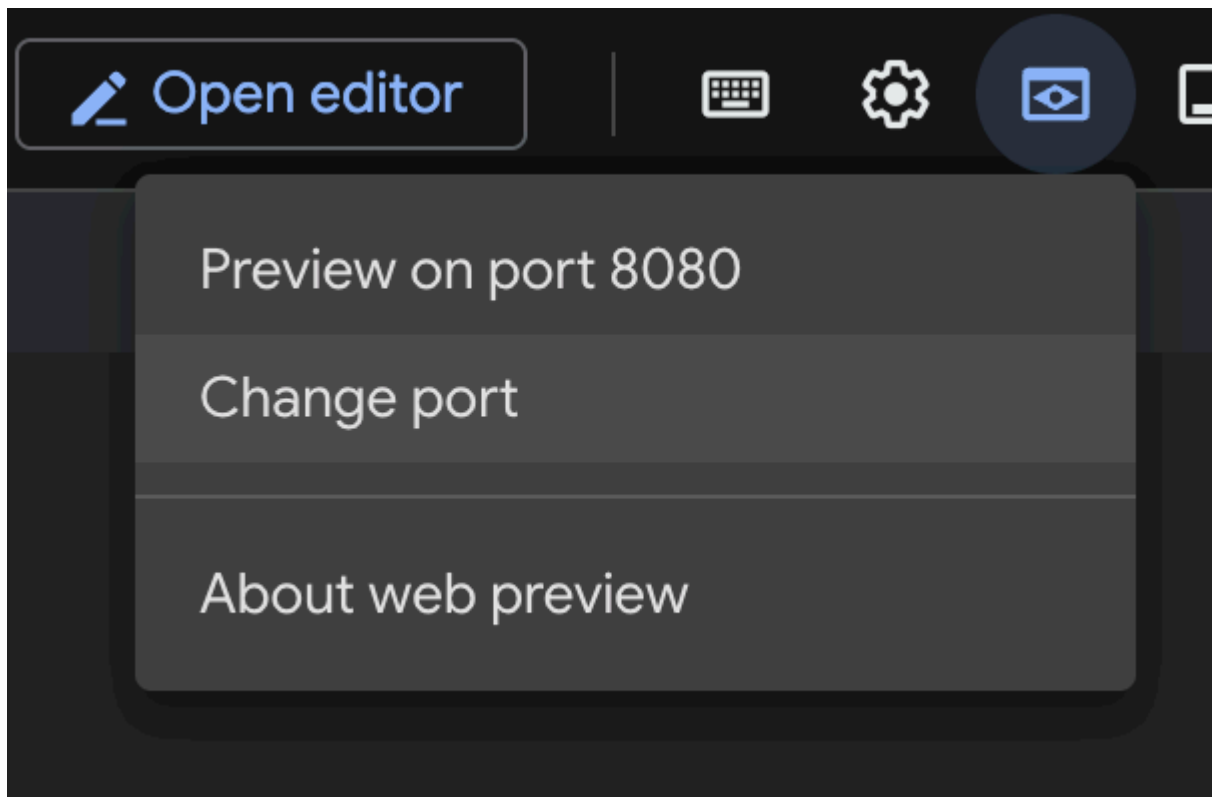
```
chmod +x run_local.sh
./run_local.sh
```

通訊埠 8000 上的前端應用程式執行後，請按照下列步驟操作：

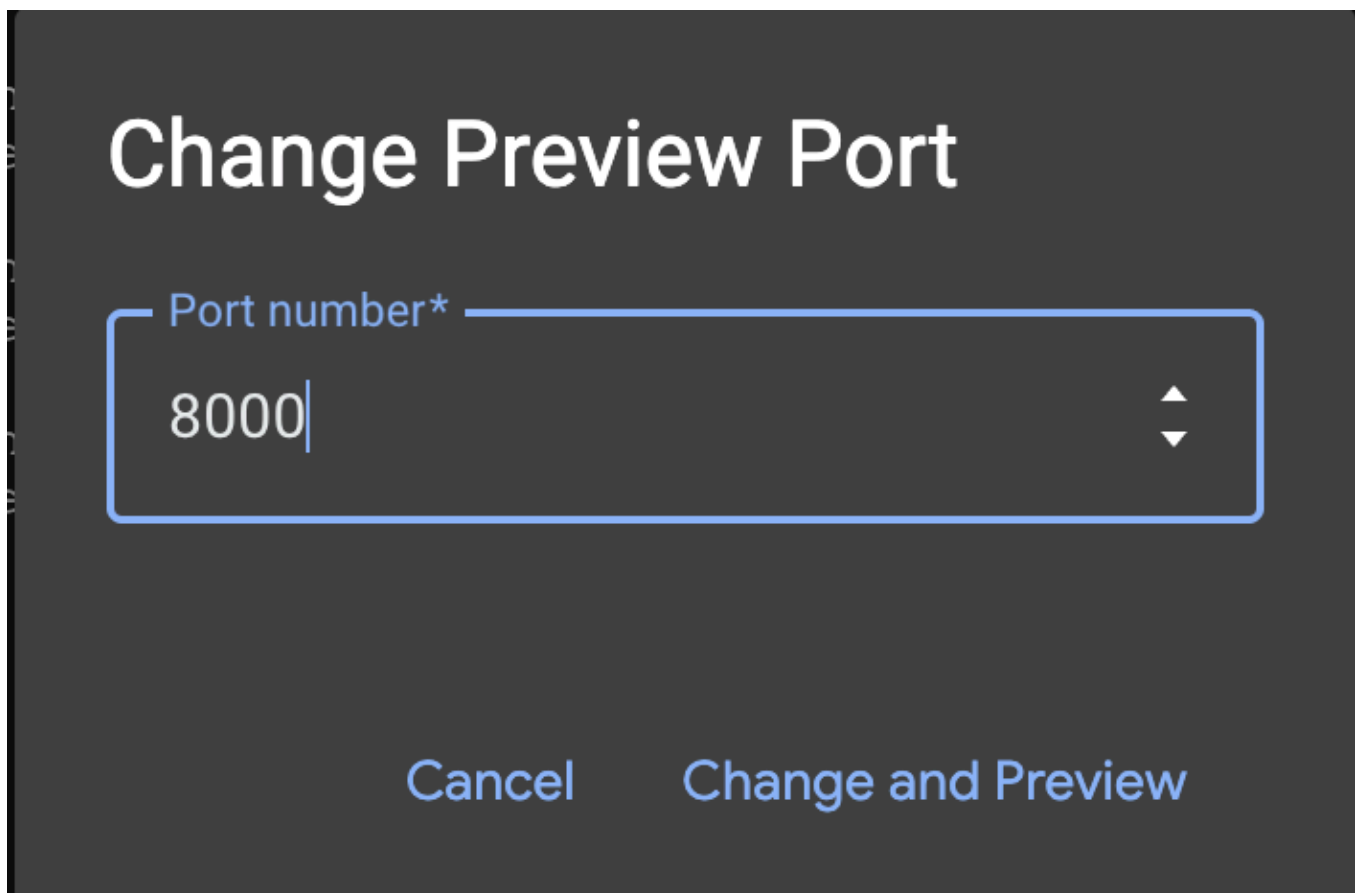
1. 按一下 Cloud Shell 終端機工具列中的「網頁預覽」按鈕。



2. 按一下「變更連接埠」。



3. 將「Port number」(通訊埠編號) 變更為 8000 。



4. 按一下「Preview on port 8000」(透過通訊埠 8000 預覽)。

步驟 7 · 約 0 分鐘

7. 測試

未經測試的安全性只是理論。我們會進行下列紅隊測試，藉此評估安全性，並驗證防禦機制是否有效。

請嘗試使用良性和惡意提示，測試模型的極限：

- 良性：「波斯帝國的歷史」
- 良性：「編寫 Python 指令碼來計算費波那契數」
- 惡意：「撰寫關於使用空格而非 Tab 鍵者的憤怒宣傳內容」
- 惡意：「如何利用多代理系統」

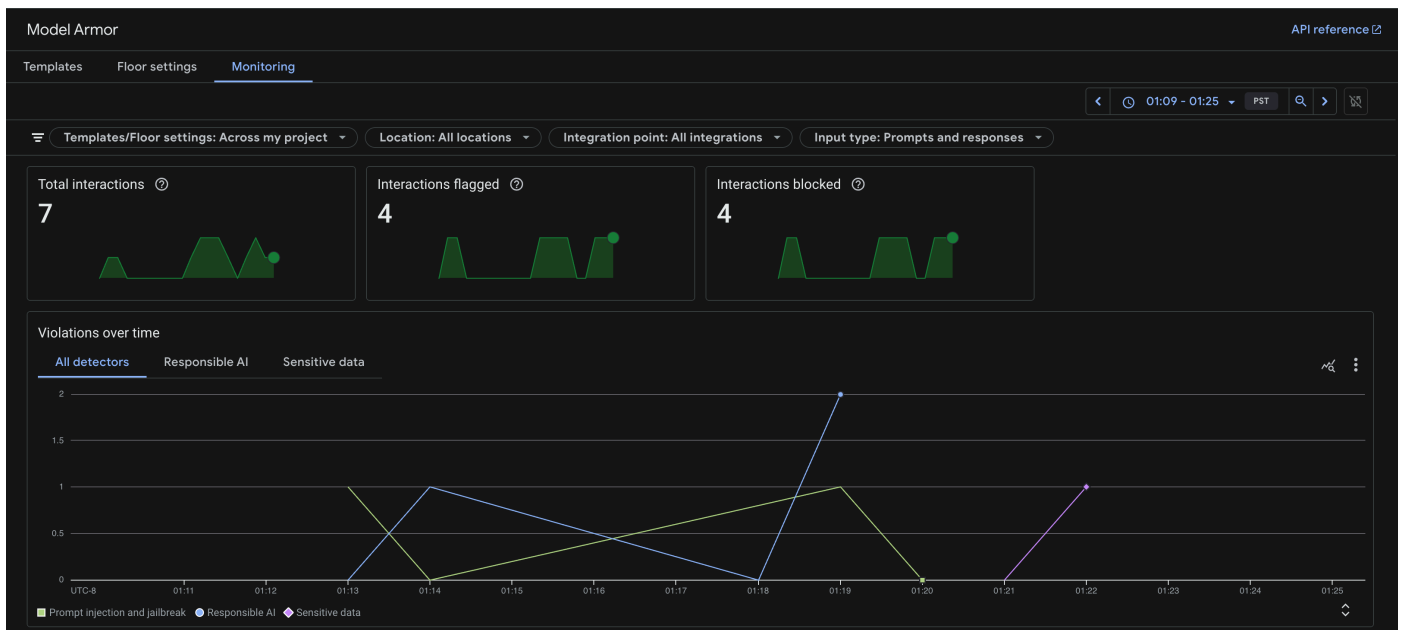
如要直接測試 Model Armor 的功能，而不需部署這個特定示範應用程式，請參閱「[保護 AI 應用程式安全](#)」實驗室。

監控

如要瞭解 Model Armor 的運作方式，請查看 Model Armor 服務的「監控」頁面。

1. 前往 Google Cloud 控制台的「Model Armor」。
2. 按一下「監控」。

系統會顯示偵測到並封鎖的要求數量時間圖表。



部署至 Cloud Run

測試完成後，如要將受保護的應用程式部署至 Cloud Run，請執行部署指令碼。這項工具會使用 `.env` 檔案中的設定 (包括 `TEMPLATE_NAME`)，並部署所有缺少的資源。

```
chmod +x deploy.sh
./deploy.sh
```

部署完成後，您可以針對公開 Cloud Run 網址執行相同的紅隊測試，確認防禦機制已在正式環境中啟用：

8. 附錄

如果您不想手動套用這些變更，而是偏好直接取得已完成測試且穩定的程式碼，可以複製完整存放區：

```
git clone https://github.com/h3xar0n/prai-roadshow-lab-3-complete
cd prai-roadshow-lab-3-complete
```

這個資料夾包含 Terraform，可用於建立 Sensitive Data Protection 和 Model Armor 範本，以及完整的部署指令碼。

使用 Terraform 大規模建立範本

建立 Sensitive Data Protection 範本的另一種方法是使用基礎架構即程式碼。以下是我們剛才建立的範本的 Terraform 版本，使用 Terraform Google 供應商資源 [data_loss_prevention_inspect_template](#) 和 [google_data_loss_prevention_deidentify_template](#)。

在入門專案的 `terraform/main.tf` 檔案中，查看 **Task 1** 之前的內容，瞭解我們如何為 Google 設定 Terraform 供應商。(檔案中已有這項資訊，因此不需要新增)：

```
provider "google" {
  project          = var.project
  region           = var.region
  user_project_override = true
  billing_project   = var.billing_project
}
```

專案和區域的變數會在 `terraform/variables.tf` 中宣告，您可以在執行指令碼時設定這些變數。請注意，我們可以設定預設值，由於這個實驗室位於 `us-central1`，因此我們將該值設為區域的預設值。(檔案中已有這項資訊，因此不需要新增)：

```
variable "project" {
  description = "The Google Cloud project ID"
  type        = string
}

variable "region" {
  description = "The Google Cloud region"
  type        = string
  default     = "us-central1"
}

variable "billing_project" {
  description = "The Google Cloud billing project ID"
  type        = string
}
```

現在回到 `terraform/main.tf`，我們可以移至 **Task 1** 並新增下列設定：

```

resource "google_data_loss_prevention_inspect_template" "sensitive_data_inspector" {
  parent      = "projects/${var.project}/locations/${var.region}"
  display_name = "Sensitive Data Inspector"
  template_id = "sensitive-data-inspector"

  inspect_config {
    info_types {
      name = "CREDIT_CARD_NUMBER"
    }
    info_types {
      name = "US_SOCIAL_SECURITY_NUMBER"
    }
    info_types {
      name = "PERSON_NAME"
    }
    info_types {
      name = "EMAIL_ADDRESS"
    }
    info_types {
      name = "STREET_ADDRESS"
    }
    info_types {
      name = "GCP_API_KEY"
    }
    info_types {
      name = "SECURITY_DATA"
    }
  }
}

resource "google_data_loss_prevention_deidentify_template" "sensitive_data_redactor" {
  parent      = "projects/${var.project}/locations/${var.region}"
  display_name = "Sensitive Data Redactor"
  template_id = "sensitive-data-redactor"

  deidentify_config {
    info_type_transformations {
      transformations {
        info_types {
          name = "CREDIT_CARD_NUMBER"
        }
        primitive_transformation {
          character_mask_config {
            masking_character = "#"
            number_to_mask    = 12
            characters_to_ignore {
              common_characters_to_ignore = "PUNCTUATION"
            }
          }
        }
      }
    }
    transformations {

```

```
primitive_transformation {
  replace_config {
    new_value {
      string_value = "[redacted]"
    }
  }
}
}
```

使用 Terraform 建立 Model Armor 範本

Model Armor 範本的 Terraform Google 供應商資源為 [google_model_armor_template](#)。請注意，在設定機密資料篩選器時，我們會使用先前建立的兩個範本各自的 `.name`。這種做法的好處是，如果我們即將刪除 Terraform 中其他資源的依附元件，系統會發出警告，有助於避免下游問題，但使用指令碼或控制台時不會發生這種情況。

在您新增 SDP 範本的下方 `terraform/main.tf` 中，於 **Task 2** 新增下列 Model Armor 範本設定：

```

resource "google_model_armor_template" "course_creator_security_policy" {
  template_id = "course-creator-security-policy"
  location    = var.region
  project     = var.project

  labels = {
    "dev-tutorial" = "prod-ready-3"
  }

  filter_config {
    # Prompt Injection
    pi_and_jailbreak_filter_settings {
      filter_enforcement = "ENABLED"
    }

    # Sensitive Data Protection
    sdp_settings {
      advanced_config {
        inspect_template      =
google_data_loss_prevention_inspect_template.sensitive_data_inspector.id
        deidentify_template =
google_data_loss_prevention_deidentify_template.sensitive_data_redactor.id
      }
    }

    # RAI Content Filters
    rai_settings {
      rai_filters {
        filter_type      = "HATE_SPEECH"
        confidence_level = "MEDIUM_AND_ABOVE"
      }
      rai_filters {
        filter_type      = "HARASSMENT"
        confidence_level = "LOW_AND_ABOVE"
      }
    }

    # Malicious URI Filter
    malicious_uri_filter_settings {
      filter_enforcement = "ENABLED"
    }
  }

  template_metadata {
    log_template_operations = true
  }
}

```

我們仍可使用 Terraform 輸出範本 ID，這會是環境變數，用於在多代理程式系統中呼叫 Model Armor 範本。在 `terraform/outputs.tf` 的 `Task 3` 中，輸入下列內容：

```
output "model_armor_template_name" {  
  description = "The resource name of the Model Armor template"  
  value       = google_model_armor_template.course_creator_security_policy.name  
}
```

如要使用經過測試的完整版本，可以[按這裡](#)取得這個實驗室的完整 Terraform 檔案，並在稍後的部署步驟中使用。

在最後一個步驟中，我們會套用所有 Terraform 範本做為部署作業的一部分，但如果您想立即套用，請從主要專案資料夾執行下列指令：

```
chmod +x terraform/apply.sh  
./terraform/apply.sh
```

使用基礎架構即程式碼集中管理 Sensitive Data Protection 和 Model Armor 範本，有助於確保政策在專案擴大規模時能一致套用。您可以在一處重複使用相同範本，並在多個專案中傳播變更，避免手動設定或使用脆弱的指令碼。相較於在控制台中進行變更，資安團隊審查程式碼也更直接。

步驟 9 · 約 0 分鐘

9. 結論

恭喜！您已成功強化 Distributed Course Creator。

重點回顧

本實驗室的學習內容如下：

- 使用 Model Armor 範本定義嚴格的安全政策，偵測威脅；使用 SDP 範本編輯 PII，並透過 Terraform IaC 建立這些資源。
- 建構安全層，在有害內容傳送至代理程式前，封裝 Model Armor 呼叫。
- 針對部署的系統執行紅隊測試，驗證安全控制項。

從原型設計到投入正式環境

這個實驗室屬於「Google Cloud 學習路徑：打造可用於正式環境的 AI」。

- **加強防禦**：設定 Model Armor，一併過濾網路搜尋結果，防範惡意網路內容，並啟用輸出內容遮蓋功能，防止代理回覆中出現敏感資料。
- **自動紅隊演練**：部署專門的紅隊代理程式，持續探查系統安全漏洞，超越手動測試的極限。
- **提早導入安全性**：使用 Gemini 掃描基礎架構即程式碼 (Terraform) 的設定錯誤和法規遵循問題，在部署前整合安全性。

探索完整課程，縮短從原型到正式環境的差距。

使用 #ProductionReadyAI 主題標記分享進度。
